



รายงานโครงงานคอมพิวเตอร์

การเปรียบเทียบการสร้างชุดทดสอบอัตโนมัติและชุดทดสอบจาก Generative AI บน Defects4J

An Empirical Comparison of Automated and Generative AI Test Generation on Defects4J

โดย

นายปวิศษ์ ประมวล	รหัสนักศึกษา 673380278-9
นายแทนคุณ พันธนิกุล	รหัสนักศึกษา 673380301-0
นายธนภูมิ จันทรา	รหัสนักศึกษา 673380272-1
นายศิμερινทร์ อุปจันทร์	รหัสนักศึกษา 673380292-5

อาจารย์ประจำวิชา

ผู้ช่วยศาสตราจารย์ ดร. ชิตสุธา สุ่มเล็ก

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353201 การประกันคุณภาพซอฟต์แวร์

ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

บทคัดย่อ

โครงการนี้เปรียบเทียบ Native IPO, Many-Independent-Objective (MIO) ใน EvoSuite, DeepSeek V4 Flash และ Gemini 3.8 Flash บน Defects4J 3.0.1-7-g8c16da82 ซึ่งมี active bugs 854 บั๊กจาก 17 โครงการ กำหนดผลหนึ่งรายการต่อบั๊กและเทคนิค รวม 3,416 ช่อง มีผลประเมิน suite 2,797 ช่อง และ 619 ช่องเป็น NO_SUITE. ผล BUG_DETECTED ต้องมีหลักฐานว่า test ล้มเหลวบน buggy version และผ่านบน fixed version.

Coverage รวมทุก modified target class ตามค่า aggregate ใน Defects4J summary. Gemini มี line/branch coverage เฉลี่ยสูงสุด 86.29%/79.54% (n=424). Native IPO ตรวจพบ 37/257 suite (14.40%), MIO 5/834 (0.60%), DeepSeek 11/853 (1.29%) และ Gemini 107/853 (12.54%).

ผลรวมสี่เทคนิคตรวจพบ 144 บั๊กไม่ซ้ำจาก 853 บั๊กที่มี suite อย่างน้อยหนึ่งเทคนิค (16.88%). สถิติ MIO budget และ token/เวลา AI แยกจาก benchmark กลางเพราะ generation logs ไม่มี run ID เชื่อมกับผลตรวจจับรายบั๊ก จึงไม่คำนวณ token ต่อบั๊กที่ตรวจพบ.

คำสำคัญ Defects4J, software testing, IPO, MIO, EvoSuite, Generative AI, code coverage, fault detection

สารบัญ

บทคัดย่อ.....	2
สารบัญ	3
สารบัญตาราง	4
สารบัญภาพ.....	4
บทที่ 1 บทนำและขอบเขตการศึกษา.....	5
บทที่ 2 แนวทางสร้างชุดทดสอบ.....	7
บทที่ 3 สภาพแวดล้อมและวิธีทดลอง	9
บทที่ 4 ผลการทดลอง	10
บทที่ 5 การวิเคราะห์และอภิปรายผล	14
บทที่ 6 สรุปผลและการทำซ้ำ	18
เอกสารอ้างอิง.....	20
ภาคผนวก ก แหล่งข้อมูลและไฟล์ทำซ้ำ.....	21
ภาคผนวก ข ตัวอย่างการตรวจย้อนกลับ	21
ภาคผนวก ค Data dictionary แบบย่อ	22

สารบัญตาราง

ตารางที่ 1	คำถามวิจัยและหลักฐานที่ใช้ตอบ	6
ตารางที่ 2	วิธีสร้าง suite ที่นำมาประเมิน	8
ตารางที่ 3	สภาพแวดล้อมการประเมิน	9
ตารางที่ 4	นิยามสถานะผลการประเมิน	10
ตารางที่ 5	สถานะและ fault detection ต่อเทคนิค; FDR ต่อ suite รวม compile/flaky ในตัวหาร	11
ตารางที่ 6	Coverage ของ modified target classes ที่มีค่าการวัด.....	12
ตารางที่ 7	ผล Wilcoxon signed-rank สำหรับ line coverage ที่วัดได้.....	13
ตารางที่ 8	จำนวนการตรวจพบทั้งหมดและการมีส่วนร่วมที่ไม่ซ้ำ	13
ตารางที่ 9	ข้อมูลการสร้าง MIO suite แยกตาม budget	15
ตารางที่ 10	สถิติจาก log การสร้างชุดทดสอบ AI	15
ตารางที่ 11	ไฟล์หลักสำหรับตรวจสอบและทำซ้ำ.....	21
ตารางที่ 12	ตัวอย่าง BUG_DETECTED ที่ตรวจ buggy/fixed และ provenance แล้ว	21
ตารางที่ 13	ความหมายฟิลด์สำหรับตรวจย้อนกลับ.....	22

สารบัญภาพ

รูปที่ 1	สัดส่วนสถานะใน suite ที่นำมาประเมิน; n ระบุได้แต่ละเทคนิค.....	11
รูปที่ 2	Line และ branch coverage เฉลี่ยของทุก modified target class ที่วัดได้; n คือจำนวนบีก.....	12
รูปที่ 3	Token consumption และ generation latency เฉลี่ยต่อ record ตาม log.....	16

บทที่ 1 บทนำและขอบเขตการศึกษา

1.1 ที่มาและความสำคัญ

การสร้าง unit test ทำได้หลายแนวทาง ตั้งแต่การออกแบบค่าป้อนเข้าแบบ combinatorial การค้นหาด้วย fitness function ไปจนถึงการใช้ Generative AI สร้างโค้ดทดสอบ. จำนวน test หรือ coverage สูงเพียงอย่างเดียวไม่ยืนยันว่า test ตรวจจับข้อบกพร่องจริง งานนี้จึงประเมิน suite จากทั้งสี่แนวทางบนโปรแกรม Java ที่มีบั๊กจริง โดยแยก code coverage ออกจากการทดสอบว่า suite แยก buggy และ fixed version ได้หรือไม่.

Defects4J จัดเตรียมบั๊กจริงและเครื่องมือ checkout เวอร์ชัน buggy/fixed

เพื่อสนับสนุนการศึกษาซอฟต์แวร์เชิงทดลอง (Just et al., 2014). งานนี้ใช้ Defects4J รุ่น 3.0.1-7-g8c16da82 ซึ่งมี 854 active bugs จาก 17 โครงการ (Defects4J contributors, n.d.) และประเมินเฉพาะ suite ที่มีหลักฐานและผ่านเกณฑ์คัดเลือกของ runner.

1.2 วัตถุประสงค์

- เปรียบเทียบ Native IPO, MIO ใน EvoSuite, DeepSeek V4 Flash และ Gemini 3.8 Flash ด้วย runner กลางชุดเดียวกัน
- วัด line และ branch coverage ในขอบเขต modified target classes พร้อมแสดงจำนวนข้อมูลที่ใช้คำนวณ
- วัด bug-level fault detection โดยตรวจผลบน buggy/fixed และแยก compile error กับ flaky/regression
- วิเคราะห์ผลรวม ensemble และข้อมูลสร้าง suite เช่น budget, seed, token และเวลา โดยรักษาที่มาของข้อมูล
- จัดเก็บ source, test suite, prompt, configuration, logs และคำสั่งทำซ้ำใน repository

1.3 ขอบเขต

Catalog มี 854 บั๊ก, 17 โครงการ, 1,073 modified target class records และ 577 คลาสไม่ซ้ำ. มี 727

บั๊กที่แก้คลาสเดียว และ 127 บั๊กที่แก้หลายคลาส. โครงการคือ Chart, Cli, Closure, Codec, Collections,

Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JxPath, Lang, Math, Mockito และ Time.

หน่วยวิเคราะห์หลักคือ project + bug_id + technique; หนึ่งบั๊กละหนึ่งครั้งต่อเทคนิค แม้มีหลายคลาสเป้าหมาย.

Coverage ใช้ aggregate counts ของทุก modified target class ที่ Defects4J เขียนใน summary.csv

จึงคำนวณจากผลรวม $\text{covered} \div \text{total}$ ของบรรทัดและ condition branches ไม่ใช่ค่าเฉลี่ยรายคลาส. จำนวน coverage n นับเฉพาะผล DONE ที่มีค่าตัวเลขจริง.

1.4 คำถามวิจัย

คำถาม	วิธีตอบ
RQ1 Coverage	เทียบ line/branch mean ของทุก modified target class ที่วัดได้ พร้อมระบุ n
RQ2 Fault detection	เทียบ BUG_DETECTED และ FDR ต่อ suite ที่รัน โดยรวม compile/flaky ในตัวหาร
RQ3 Ensemble	นับบั๊กไม่ซ้ำอย่างน้อยหนึ่งเทคนิคตรวจพบ และส่วนร่วมเฉพาะของแต่ละเทคนิค
RQ4 MIO budget	สรุป coverage criterion รวมและเวลาแยกตาม budget พร้อม seed และข้อจำกัด
RQ5 AI generation	เปรียบเทียบ token/เวลาเฉลี่ยต่อ generation record แยกจาก benchmark detection

ตารางที่ 1 คำถามวิจัยและหลักฐานที่ใช้ตอบ

บทที่ 2 แนวทางสร้างชุดทดสอบ

2.1 Native IPO และ pairwise testing

Pairwise testing ต้องการให้ทุกคู่ค่าของ input factors ปรากฏอย่างน้อยหนึ่งครั้ง. IPO (In-Parameter-Order) สร้าง covering array แบบเพิ่มพารามิเตอร์ทีละตัว โดยขยายแถวเดิมในแนวนอนและเพิ่มแถวในแนวตั้งเมื่อยังมีคู่ค่าที่ขาด (Lei & Tai, 1998). Pairwise ช่วยควบคุมจำนวนกรณี แต่ไม่รับประกันการตรวจพบบั๊ก จึงวัด fault detection แยกจาก pair coverage.

สาย IPO ใช้ Native IPO pipeline ใน `Combinatorial\_IPO/Code/` วิเคราะห์ Java API ของ modified target class เพื่อระบุ constructor, method, receiver และ input factors แล้วใช้ type adapters กำหนดวิธีสร้างค่าและ value domains. คลาสที่มีทางสร้างอินพุตและเรียกเป้าหมายได้ตามเกณฑ์ของตัววิเคราะห์จึงได้รับสถานะ `AUTO\_READY`; กรณีที่ยังขาด adapter, public entry point หรือใช้ pairwise ไม่ได้จะบันทึกเหตุผลใน audit และ routing manifest แทนการนับเป็น suite ที่พร้อมประเมิน.

สำหรับ callable ที่มี input factors อย่างน้อยสองตัว อัลกอริทึมเริ่มจากการจับคู่ค่าของสอง factors แรก แล้วเพิ่ม factor ทีละตัวด้วย horizontal growth ซึ่งเติมค่าลงในแถวเดิมเพื่อครอบคลุมคู่ใหม่ และ vertical growth ซึ่งเพิ่มแถวเมื่อยังมีคู่ที่ขาด. Mandatory seed rows ถูกเพิ่มเป็นกรณีบังคับแยกจากชั้นสร้าง covering array. ตัวตรวจ pairwise อีกระยะตรวจว่าแต่ละ method ครอบคลุม expected pairs ครบก่อนสร้าง JUnit 4 suite; suite จะเผยแพร่เป็น `FIXED\_VERIFIED` เมื่อคอมไพล์และผ่านการรันบน Defects4J fixed version. `verified\_suites\_manifest.json` บันทึก target class, suite path และ SHA-256.

ตัวอย่าง Lang-1: manifest ของ `NumberUtils.toInt(String,int)` ระบุ pair coverage 55/55 คู่ และ suite ระดับคลาสผ่าน fixed-version verification (`OK (666 tests)`). อย่างไรก็ตาม benchmark กลางจำแนก Lang-1 เป็น `NOT\_DETECTED` โดยวัด line coverage 22.93% และ branch coverage 11.24%. ตัวอย่างนี้แสดงว่า pair coverage ของอินพุตและการผ่าน fixed-version verification ไม่ได้แปลว่าจะตรวจจับบั๊กได้.

PICT เป็นข้อมูลอ้างอิงของสาย IPO ไม่ใช่เทคนิค IPO ที่นำมาเปรียบเทียบใน benchmark นี้. ตัวเลข 173 suites และ 42,398 test cases มาจาก baseline รอบก่อนใน

`Combinatorial\_IPO/Results/baseline/baseline\_manifest.json`; ไม่ใช่จำนวน suite หรือผลวัดของ Native IPO ใน snapshot ปัจจุบัน.

2.2 MIO และ EvoSuite

MIO เป็น search algorithm สำหรับการสร้าง test suite ที่จัดการเป้าหมาย coverage

หลายรายการแยกอิสระและปรับการค้นหตาม budget (Arcuri, 2018). EvoSuite สร้าง unit tests สำหรับ Java ด้วย search-based approaches และ assertions (Fraser & Arcuri, 2011).

Budget experiment ใช้ EvoSuite 1.0.6, algorithm MIO, criterion 'LINE:BRANCH', budgets 30/60/120 วินาที และ seeds 101, 102, 103. สคริปต์ทำ 3×3 รวม 9 runs ต่อ target class. ผล generation แยกจากผล benchmark กลาง. Raw criterion ให้ coverage ค่าเดียว จึงรายงานเป็น combined coverage ไม่แยก line และ branch.

2.3 DeepSeek และ Gemini

สคริปต์ 'scripts/kku_generate.py' เรียก KCU IntelSphere API โดยใช้ default identifiers 'deepseek-v4-flash' และ 'gemini-3.8-flash'. ชื่อโมเดลในรายงานเป็น labels ที่บันทึกในโครงการ; generation records ไม่เก็บ provider model ID ทุกแถว จึงไม่อนุมานสถาปัตยกรรมเฉพาะรุ่น.

Prompt ใช้ source code ของ target class และ defect context ที่มีใน repository. Prompt กำหนด JUnit 4, package ให้ตรงกับโครงการ, test timeout, หลีกเลี่ยง dependency mocking ภายนอก และชี้แนะให้ทดสอบค่าขอบเขต. Prompt ที่ใช้จริงเก็บราย bug ใน 'Deepseek-v4_flash/Prompt/' และ 'Gemini-3_8_flash/Prompt/'.

แนวทาง	เครื่องมือ/วิธี	หลักฐานการสร้าง
Combinatorial	Native IPO, 2-way pair coverage	Generator source, verified manifest, suite hash
Search-based	EvoSuite 1.0.6, MIO	Budget/seed summary และ raw reports
Generative AI	DeepSeek V4 Flash label	Prompt, JUnit test, generation records
Generative AI	Gemini 3.8 Flash label	Prompt, JUnit test, generation records

ตารางที่ 2 วิธีสร้าง suite ที่นำมาประเมิน

บทที่ 3 สภาพแวดล้อมและวิธีทดลอง

3.1 แหล่งข้อมูลและการจัดการผล

Catalog อยู่ใน `target\_benchmark/all\_bugs\_catalog.json`; master results อยู่ใน `results/master\_benchmark\_summary.csv`; descriptive statistics อยู่ใน `results/master\_descriptive\_stats.json`. Master มี 3,416 คีย์ไม่ซ้ำ ครอบคลุม 854 บั๊ก × 4 เทคนิค. มี 2,797 คีย์ที่ runner ประเมินแล้ว; 619 คีย์เป็น NO_SUITE; unresolved outcomes 0. บั๊กที่ตรวจเพิ่มเติมใน `results/suite\_gap\_audit.csv` แยกกรณีไม่มีไฟล์ Javaออกจากไฟล์ผู้สมัครที่ยังไม่ผ่าน manifest หรือ target matcher.

Runner ตรวจ target classes กับ Defects4J, เลือก suite ตาม bug key และเทคนิค และเก็บ SHA-256, run ID, timestamp, duration และ structured log. ผลมาตรฐานหนึ่งรายการต่อคีย์ถูกรวมใน master; run ซ้ำไม่ได้เพิ่มจำนวนบั๊กในตัวหาร.

3.2 สภาพแวดล้อม

องค์ประกอบ	การตั้งค่าที่อ้างอิง
Container	Docker service `defects4j_sqa`; timezone Asia/Bangkok
Benchmark	Defects4J 3.0.1-7-g8c16da82; 854 active bugs ตามรุ่นที่ติดตั้ง
Java	ติดตั้ง OpenJDK 8 และ 11; default JAVA_HOME เป็น JDK 11; MIO batch ตั้ง JDK 8
Test framework	JUnit 4; test prompt กำหนด timeout 4 วินาที
Coverage	Defects4J coverage workflow พร้อม Cobertura summary

ตารางที่ 3 สภาพแวดล้อมการประเมิน

Dockerfile และ compose อยู่ใน `docker/Dockerfile` และ `docker/docker-compose.yml`. Defects4J CLI ใช้ JDK 11; MIO generation บางขั้นตอนตั้ง JDK 8. Dockerfile สร้างจาก Ubuntu 20.04 พร้อมตรึง commit ของ Defects4J และ PICT จึงไม่ต้องเตรียม image พื้นฐานเฉพาะของเครื่องก่อน build. การ build ครั้งแรกต้องดาวน์โหลดและ initialize catalog ของ Defects4J ซึ่งใช้พื้นที่และเวลามาก; แพ็กเกจระบบยังติดตั้งจาก Ubuntu package archive.

3.3 ขั้นตอนประเมิน

1. อ่าน target classes จาก catalog และตรวจเทียบกับ Defects4J
2. ค้นหา suite ตาม project, bug และ technique; หากไม่มีให้บันทึก NO_SUITE
3. คอมไพล์ suite และวัด coverage ผ่าน Defects4J workflow บน buggy version
4. รัน suite บน buggy; ตรวจสอบ fixed version ด้วย suite เดียวกันเพื่อยืนยัน defect-specific failure
5. บันทึก status, filename, suite hash, run ID, timestamp, duration และ log
6. รวมผลเป็น bug-technique matrix แล้วคำนวณสถิติจากคีย์ไม่ซ้ำ

3.4 นิยามสถานะและตัวชี้วัด

สถานะ	นิยามที่ใช้
BUG_DETECTED	test อย่างน้อยหนึ่งกรณี fail บน buggy และไม่มี failure บน fixed; ตรวจสอบจาก log
NOT_DETECTED	suite รันได้แต่ไม่ทำให้ buggy fail
FLAKY_OR_REGRESSION	failure ยังเกิดบน fixed หรือไม่จำเพาะต่อบั๊ก
COMPILE_ERROR	suite คอมไพล์ไม่ผ่าน; ไม่ถือเป็น coverage 0 ที่วัดได้
NO_SUITE	ไม่มี suite ให้ประเมิน; ไม่ถือเป็น NOT_DETECTED

ตารางที่ 4 นิยามสถานะผลการประเมิน

FDR ต่อ suite = $\text{BUG_DETECTED} \div \text{suite}$ ที่นำมาประเมิน; compile errors และ flaky/regression อยู่ในตัวหาร. อัตรา detected $\div 854$ แสดงเพิ่มเติมเพื่อสื่อสัดส่วนการตรวจพบใน catalog ทั้งหมด; NO_SUITE ไม่ถูกเรียกว่าเป็นผลทดสอบ.

บทที่ 4 ผลการทดลอง

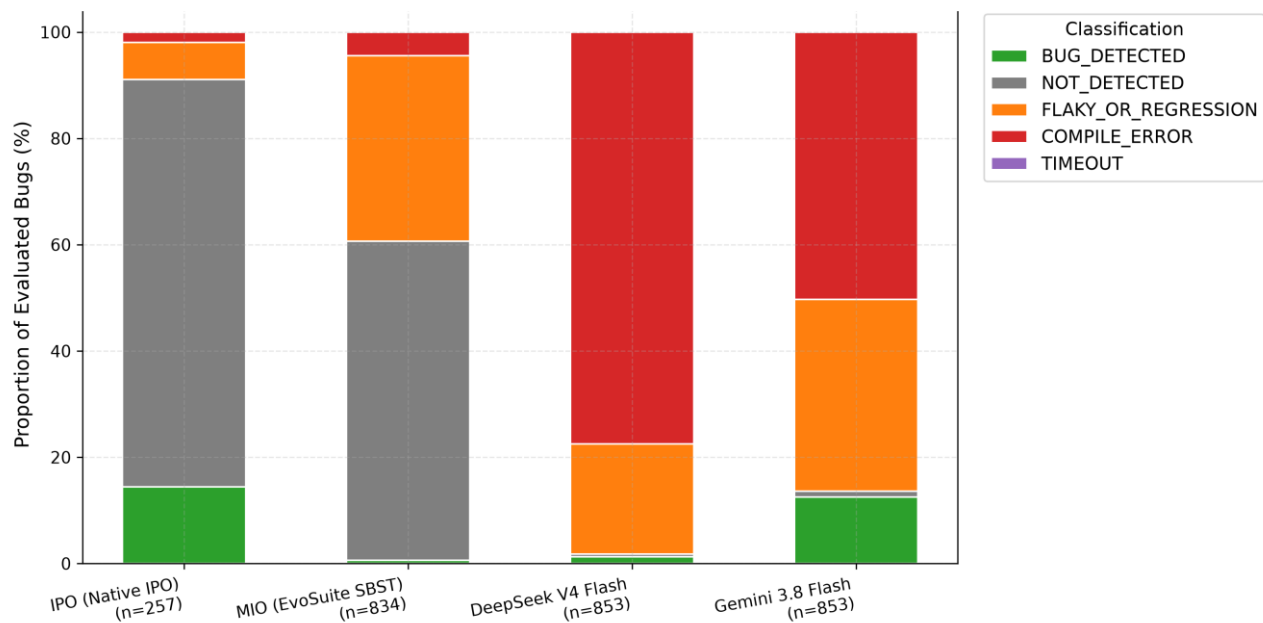
4.1 ความครบถ้วนและสถานะผล

Catalog มี $854 \text{ บั๊ก} \times 4 \text{ เทคนิค}$ รวม 3,416 คีย์ไม่ซ้ำ. มี suite และผลประเมิน 2,797 รายการ; 619 รายการไม่มี suite. สถานะมี 1,665 DONE, 1,132 COMPILE_ERROR และ 0 TIMEOUT. ในผลที่รันได้มี 160 BUG_DETECTED, 711 NOT_DETECTED และ 794 FLAKY_OR_REGRESSION; unresolved outcomes 0.

เทคนิค	มี suite	NO_SUITE	DONE	compile	พบ defect	flaky	ไม่พบ	FDR suite	พบ/854
Native IPO	257/854	597	252	5	37	18	197	14.40%	4.33%
MIO ใน EvoSuite	834/854	20	797	37	5	291	501	0.60%	0.59%
DeepSeek V4 Flash	853/854	1	192	661	11	177	4	1.29%	1.29%
Gemini 3.8 Flash	853/854	1	424	429	107	308	9	12.54%	12.53%

ตารางที่ 5 สถานะและ fault detection ต่อเทคนิค; FDR ต่อ suite รวม compile/flaky ในตัวหาร

‘DONE’ หมายถึงการประเมินสิ้นสุดโดยไม่เกิด compile error; fault status ของแถว DONE ยังอาจเป็น flaky/regression. จำนวน 619 NO_SUITE แสดงแยกจาก NOT_DETECTED.



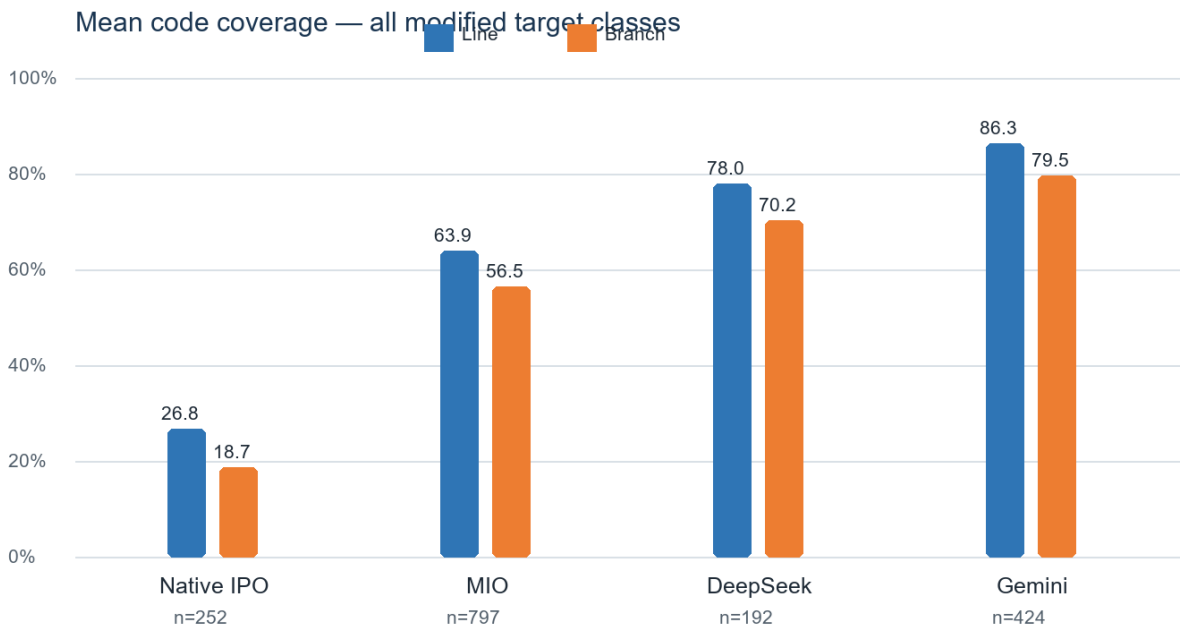
รูปที่ 1 สัดส่วนสถานะใน suite ที่นำมาประเมิน; n ระบุได้แต่ละเทคนิค

4.2 Coverage ของ modified target classes

Defects4J coverage workflow เขียนจำนวนบรรทัดและ condition branches ที่ครอบคลุมและทั้งหมดเป็น aggregate summary ของคลาสที่ถูกวัด. รายงานจึงรวมบักที่แก้หนึ่งคลาสและหลายคลาส โดยใช้คำร้อยละที่คำนวณจาก aggregate counts. ค่า n ต่างกันเพราะ suite ที่ไม่มีหรือ compile ไม่ผ่านไม่มี coverage ที่วัดได้. SD แสดงการกระจายระหว่างบัก ไม่ใช่ standard error.

เทคนิค	n	Line mean $\pm$ SD	Branch mean $\pm$ SD
Native IPO	252	26.76 $\pm$ 21.50%	18.67 $\pm$ 19.33%
MIO ใน EvoSuite	797	63.85 $\pm$ 34.26%	56.51 $\pm$ 35.94%
DeepSeek V4 Flash	192	78.02 $\pm$ 30.80%	70.22 $\pm$ 31.10%
Gemini 3.8 Flash	424	86.29 $\pm$ 25.10%	79.54 $\pm$ 26.94%

ตารางที่ 6 Coverage ของ modified target classes ที่มีค่าการวัด



รูปที่ 2 Line และ branch coverage เฉลี่ยของทุก modified target class ที่วัดได้; n คือจำนวนบั๊ก

Gemini มี coverage mean สูงสุดในผลที่วัดได้. ค่าเฉลี่ยนี้ยังมี selection bias ได้ เพราะ coverage คำนวณได้เฉพาะกรณีที่ suite ผ่านการ compile และทำงานสำเร็จ; Gemini compile error 429/853 และ DeepSeek 661/853.

4.3 การเปรียบเทียบ line coverage แบบจับคู่

คู่เปรียบเทียบ	บั๊กที่มีคู่ข้อมูล	p หลัง Holm	Rank-biserial
Gemini 3.8 Flash – DeepSeek V4 Flash	136	4.92e-12	0.834
MIO ใน EvoSuite – Gemini 3.8 Flash	399	7.28e-33	-0.759
MIO ใน EvoSuite – DeepSeek V4 Flash	183	1.75e-02	-0.222

คู่เปรียบเทียบ	บั๊กที่มีคู่ข้อมูล	p หลัง Holm	Rank-biserial
MIO ใน EvoSuite – Native IPO	245	1.64e-38	0.993
Gemini 3.8 Flash – Native IPO	147	1.90e-24	1.000
DeepSeek V4 Flash – Native IPO	69	4.92e-12	1.000

ตารางที่ 7 ผล Wilcoxon signed-rank สำหรับ line coverage ที่วัดได้

เปรียบเทียบเฉพาะ project-bug ที่ทั้งสองเทคนิคมี line coverage และปรับ p-value ด้วย Holm สำหรับหาคู่. ค่า rank-biserial เป็นบวกเมื่อเทคนิคซ้ายมี coverage สูงกว่า; ผลนี้เป็นการเปรียบเทียบเชิงสำรวจของ suite ที่ผ่านการวัดและไม่ลบข้อจำกัดจากจำนวน compile error ที่ต่างกัน.

4.4 Ensemble

Union ของ BUG_DETECTED มี 144 บั๊กไม่ซ้ำจาก 853 บั๊กที่มี suite อย่างน้อยหนึ่งเทคนิค (16.88%) และคิดเป็น 16.86% ของ 854 บั๊ก. นี่เป็นการรวมผลที่วัด ไม่ใช่การรัน ensemble suite ใหม่.

เทคนิค	พบทั้งหมด	พบเฉพาะเทคนิค
Native IPO	37	27
MIO ใน EvoSuite	5	5
DeepSeek V4 Flash	11	5
Gemini 3.8 Flash	107	91

ตารางที่ 8 จำนวนการตรวจพบทั้งหมดและการมีส่วนร่วมที่ไม่ซ้ำ

บทที่ 5 การวิเคราะห์และอภิปรายผล

5.1 Coverage และ fault detection

Gemini มีค่า coverage สูงสุดในชุดผลที่วัดได้ ขณะที่ Native IPO มี FDR ต่อ suite สูงสุด. MIO และ DeepSeek แสดงว่าการเข้าถึงโค้ดไม่ได้รับประกันว่า assertion จะจับพฤติกรรมบกพร่องได้. Coverage เป็นตัวชี้วัดการเข้าถึงโค้ด; fault detection วัดว่า suite แยก buggy จาก fixed ได้หรือไม่.

Native IPO ตรวจพบ 37/257 suite (14.40%); ต่อ catalog เท่ากับ 37/854 (4.33%). มี suite ให้ประเมิน 257 บั๊ก. Gemini ตรวจพบ 107/853 (12.54%) และมี compile error 429 รายการ. การรายงานทั้งตัวหาร suite และตัวหาร catalog ป้องกันการอ่านเปอร์เซ็นต์โดยไม่เห็นขอบเขต.

สถานะ 'FIXED_VERIFIED' เป็นผลตรวจขั้นสร้าง suite ไม่ใช่ผลตรวจจับบั๊กของ benchmark กลาง. manifest IPO มี 277 suites ระดับคลาส ครอบคลุม 257 bug IDs; เมื่อนำเข้าตัวรันกลางพบ 'BUG_DETECTED' 37 รายการ, 'COMPILE_ERROR' 5 รายการ และ 'FLAKY_OR_REGRESSION' 18 รายการ. จึงต้องอ่านสถานะจากสองขั้นตอนแยกกัน.

MIO ตรวจพบ 5 จาก 834 suites ที่นำมาประเมิน (0.60%); ทั้ง 5 เป็นการตรวจพบเฉพาะ MIO ในชุดข้อมูลนี้. ยังมี 291 'FLAKY_OR_REGRESSION' และ 37 'COMPILE_ERROR'. รูปแบบนี้สอดคล้องกับความท้าทายด้าน test oracle และความเสถียรของ suite แต่จำนวนสถานะเพียงอย่างเดียวไม่ยืนยันว่า assertion ที่สร้างจาก buggy version เป็นสาเหตุ จึงไม่สรุปกลไกดังกล่าวเป็นข้อเท็จจริง.

5.2 Compile errors และความไม่เสถียร

ใน suite evaluations 1,132 รายการ compile ไม่ผ่าน และ 794 รายการเป็น flaky/regression. DeepSeek มี compile errors 661/853 และ Gemini 429/853. สถานะเหล่านี้ไม่ถูกแทนด้วย coverage 0; FDR ต่อ suite รวมไว้ในตัวหารเพื่อให้เห็นความพร้อมใช้จริง.

5.3 MIO search budget

MIO budget summary เก็บผลตาม target class, budget และสาม seeds. Raw EvoSuite criterion 'LINE;BRANCH' ให้ coverage ค่าเดียว; สคริปต์บันทึกค่าเดียวกันในคอลัมน์ line และ branch ดังนั้นรายงานเป็น combined coverage และไม่แสดง branch แยกหรือ mutation score ซึ่งใน summary เป็นค่าคงที่ placeholder. n คือจำนวน class-budget records ไม่ใช่จำนวนบั๊กไม่ซ้ำ.

Budget	records	Coverage รวม mean $\pm$ SD	SD ใน 3 seeds เฉลี่ย	เวลาเฉลี่ย	เทียบก่อน
30 วินาที	1023	65.73 $\pm$ 31.96%	3.26 จุด	68.29 วินาที	ฐาน
60 วินาที	1015	68.73 $\pm$ 31.29%	2.66 จุด	90.62 วินาที	+3.00 จุด
120 วินาที	989	70.82 $\pm$ 30.40%	2.42 จุด	194.41 วินาที	+2.09 จุด

ตารางที่ 9 ข้อมูลการสร้าง MIO suite แยกตาม budget

SD coverage ในตารางแสดงการกระจายระหว่าง class-budget records; SD ใน seed column คือค่าเฉลี่ยของความผันผวนภายใน record จากสาม seeds. เวลาเฉลี่ยรวม setup/compile จึงยาวกว่า search budget ได้. การทดสอบ Wilcoxon สำหรับ MIO จับคู่เฉพาะ project-bug-target class ที่มีผลทั้งสอง budget และปรับ p-value ด้วย Holm. ผล budget เป็น generation experiment แยกจาก benchmark กลางและใช้เป็นคำอธิบายเชิงพรรณนา ไม่ใช่ FDR.

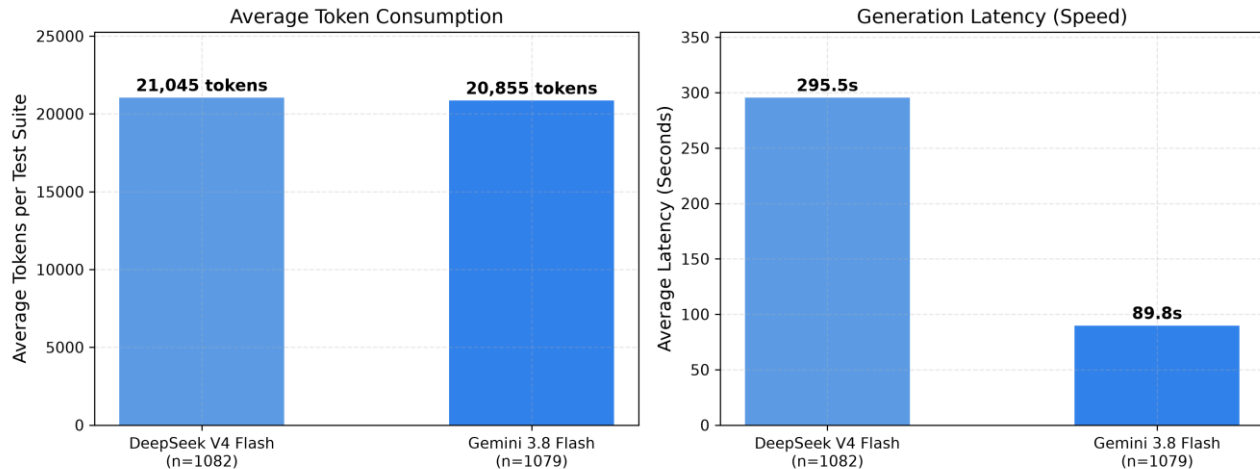
เมื่อจับคู่ target classes ที่มีผลทั้งสอง budget พบความต่างของ coverage ระหว่าง 30 กับ 60 วินาที ($n=1006$, p หลัง Holm= $3.39e-84$, rank-biserial= -0.828) และระหว่าง 60 กับ 120 วินาที ($n=981$, p หลัง Holm= $1.54e-67$, rank-biserial= -0.765). rank-biserial ติดลบเพราะคำนวณผลต่าง budget สั้นลบ budget ยาว; คำนี้นี้จึงชี้ว่า coverage ใน budget ยาวสูงกว่าในคู่ที่วัดได้.

เมื่อเทียบค่าเฉลี่ยของ records ในตาราง Coverage เพิ่ม 3.00 จุดร้อยละจาก 30 เป็น 60 วินาที และเพิ่มอีก 2.09 จุดร้อยละจาก 60 เป็น 120 วินาที; เวลา generation เฉลี่ยเพิ่มจาก 68.29 เป็น 90.62 และ 194.41 วินาทีตามลำดับ. ผล paired Wilcoxon แสดงความแตกต่างใน target classes ที่มีข้อมูลครบทั้งสอง budget แต่จำนวน records และกลุ่มเป้าหมายของแต่ละ budget ไม่เท่ากัน. จึงเป็น trade-off ที่สังเกตได้ในชุดข้อมูลนี้ ยังไม่เพียงพอจะประกาศว่า 60 วินาทีเป็นค่าที่ดีที่สุดทั่วไปหรือสรุปความคุ้มค่าใน CI/CD.

5.4 AI token และเวลา generation

โมเดลตาม label	records	Token/record เฉลี่ย	เวลา/record เฉลี่ย
Gemini 3.8 Flash	1079	20,855.06	89.78 วินาที
DeepSeek V4 Flash	1082	21,045.26	295.54 วินาที

ตารางที่ 10 สถิติจาก log การสร้างชุดทดสอบ AI



รูปที่ 3 Token consumption และ generation latency เฉลี่ยต่อ record ตาม log

Generation record ไม่ใช่ benchmark evaluation และไม่มี run ID ผูกกับผลตรวจพบรายบัก จึงห้ามใช้หารเป็น token ต่อการตรวจพบ. Provider model ID ไม่ได้เก็บครบทุกแถว จึงรายงานชื่อรุ่นตาม label ในโครงการ.

5.5 ข้อจำกัด

- จำนวน suite ต่างกันระหว่างเทคนิค; 619 ช่อง NO_SUITE เป็นช่องที่ไม่มี suite สำหรับ benchmark evaluation และไม่ใช่ผลลบของการทดสอบ. IPO มี 597 ช่อง: 37 GENERATION_OR_VERIFICATION_ERROR และ 560 SKIPPED_NOT_READY. ใน 37 ช่องมี candidate Java 6 ช่องที่ fixed-version verification ไม่ผ่าน และอีก 31 ช่องไม่พบ candidate; ใน 560 ช่องที่ถูกข้าม มี 495 NEEDS_ADAPTER, 47 NEEDS_ENTRY_POINT, 16 ที่มีทั้งสองสาเหตุ และ 2 ที่มี NEEDS_ADAPTER ร่วมกับ NOT_PAIRWISE_APPLICABLE.
- จำนวนระดับคลาสของ IPO เป็นคนละหน่วยกับจำนวน bug-technique: catalog มี modified target records 1,073 รายการ โดย 3 รายการของ Codec-14 เป็นไฟล์ทรัพยากร '.txt' จึงมี Java class instances 1,070 รายการ. Audit ระบุ 319 'AUTO_READY'; generation manifest มี 277 'FIXED_VERIFIED', 42 'GENERATION_OR_VERIFICATION_ERROR' และ 751 'SKIPPED_NOT_READY'. ชุดที่ verified ครอบคลุม 2,224 methods และ 109,700 test cases ใน 15 โปรเจกต์ และครอบคลุม 257 bug IDs. ยื่อนำจำนวนระดับคลาสเหล่านี้ไปบวกกับ 597 ช่อง 'NO_SUITE' ซึ่งนับระดับบัก-เทคนิค.
- MIO มี 20 'GENERATION_FAILURE' ระดับ bug-technique; batch runner ทำ 3 budgets × 3 seeds หรือ 9 runs ต่อ target class. สาเหตุคือ Mockito 15 ช่อง compile ไม่ผ่านจาก Bintray/JCenter; Math-13, Math-31 และ Gson-3 เกิด EvoSuite 1.0.6 internal NPE; Gson-8 เกิด JVM native crash; JacksonDatabind-24

เกิดปัญหา character encoding ตอน compile. ทุกช่องยังคงเป็น `NO\_SUITE` ใน benchmark เพราะไม่มี suite สำหรับประเมิน.

- Member 3 ส่งไฟล์ AI 31 ไฟล์สำหรับ 29 คีย์บัก-เทคนิค; runner ประเมินครบแล้ว ได้ COMPILE_ERROR 26 คีย์ และ DONE 3 คีย์ โดย 3 คีย์ในกลุ่ม DONE ยัง fail บน fixed. เหลือ AI NO_SUITE 2 คีย์ที่ target mismatch (Math-13); catalog/Defects4J ระบุ target
org.apache.commons.math3.optimization.general.AbstractLeastSquaresOptimizer แต่ candidate ใช้ package org.apache.commons.math3.optim.nonlinear.vector.jacobian.
- Coverage aggregate รวม modified target classes จาก Defects4J summary; การเปลี่ยนนี้ยังขึ้นกับชุด suite ที่ compile และรันสำเร็จ.
- Compile errors และ flaky/regression สูงใน AI suites; coverage mean จากชุดที่วัดได้อาจมี survivorship bias.
- MIO budget summary ใช้ criterion coverage รวม ไม่แยก line/branch; cohort ต่อ budget ไม่เหมือนกัน.
- AI generation log ไม่มี run ID ครบ; ไม่รองรับการวิเคราะห์ token/time ต่อผล detection.
- Benchmark evaluation ต่อ bug-technique key ไม่ได้ทำซ้ำทุก suite หลายรอบเพื่อวัด variance.
- ข้อมูลสรุปจำกัดอยู่ที่ Java Defects4J catalog และ suites ที่ทีมส่งมอบใน snapshot นี้.

บทที่ 6 สรุปผลและการทำซ้ำ

6.1 สรุปคำตอบ

- RQ1: Gemini มี line/branch coverage mean สูงสุดในผลที่วัดได้เมื่อรวม modified target classes ทุกคลาส; n แสดงได้ตารางและผล compile error แสดงแยก.
- RQ2: Native IPO มี FDR ต่อ suite สูงสุดที่ 14.40%; Gemini ตรวจพบจำนวนบั๊กมากที่สุด 107/853 (12.54%). ตัวหารต่างกัน.
- RQ3: Union พบ 144 บั๊กไม่ซ้ำ; ส่วนร่วมเฉพาะคือ Gemini 91, IPO 27, MIO 5 และ DeepSeek 5.
- RQ4: Coverage criterion รวมใน MIO summary เพิ่มตาม budget เชิงพรรณนา; cohort ต่างกันจึงไม่ยืนยัน causal gain หรือจุดอิมิตัว.
- RQ5: AI logs ให้ token และเวลาเฉลี่ยต่อ generation record; ไม่มี run ID เชื่อมกับ detection จึงไม่คำนวณต้นทุนต่อบั๊ก.

6.2 ข้อสรุป

ไม่มีวิธีเดียวที่ดีที่สุดทุกตัวชี้วัด. Gemini ให้ coverage สูงสุดในผลที่วัดได้และตรวจพบ 107 บั๊ก แต่มี compile errors จำนวนมาก. Native IPO มี FDR ต่อ suite สูงสุดใน snapshot นี้ แต่มี suite ผ่านเกณฑ์ประเมินน้อยกว่า. ผล MIO และ DeepSeek แสดงว่า coverage สูงไม่ได้รับประกัน fault detection. การเลือกใช้จึงควรพิจารณา coverage, detection, ความสามารถ compile และ stability ร่วมกัน.

6.3 งานต่อไป

- Member 3 ตรวจ Math-13: แก้ package ให้ตรงกับ classes.modified ที่ Defects4J ยืนยัน หรือยืนยันว่าไม่มี suite ใช้ได้เพื่อคงสถานะ NO_SUITE; หากส่งไฟล์ใหม่ Member 4 ประเมินเฉพาะคีย์ที่เปลี่ยน.
- บันทึก immutable model ID และ generation record ID เชื่อมกับ suite hash และ benchmark run ID.
- แยก MIO raw reports ตาม class, budget และ seed ให้มี provenance ครบทุกแถว; แยก coverage criterion ที่วัดได้จริง.
- ปรับ prompt และเพิ่ม compile feedback เพื่อแก้ compile errors และ flaky assertions.
- ทำ repeated benchmark executions หากต้องการรายงาน variance ของผลต่อ suite.

6.4 คำสั่งทำซ้ำ

คำสั่งต่อไปนี้อ้างอิง README และ Docker configuration. การประเมินครบทุก suite ใช้เวลาตามจำนวน suite และ environment; หากต้องการสร้าง analytics จากผลที่บันทึกไว้สามารถข้าม runner ได้.

```
docker compose -f docker/docker-compose.yml up -d --build
docker exec defects4j_sqa python3 /workspace/scripts/run_benchmark.py --all-bugs --resume
python scripts/consolidate_master_results.py
python scripts/advanced_data_analytics.py
```

คำสั่งต่อไปนี้ใช้ทำซ้ำการสร้าง suite ทั้งหมด: IPO เรียก unified pipeline ส่วน MIO ใช้ตัวเลือก `--all` ตาม parser ของ batch runner และรัน 9 ครั้งต่อ target class (3 budgets × 3 seeds).

```
docker exec defects4j_sqa bash /workspace/Combinatorial_IPO/run_batch.sh
docker exec defects4j_sqa python3 /workspace/MIO_Algorithm/Code/batch_evosuite.py --all
```

การรัน AI ต้องตั้ง KKU API key ใน environment ตาม `.env.example`; ห้ามใส่ secret ใน repository. ตัวเลขในเล่มอ้างอิง snapshot วันที่ 26 กันยายน 2569.

เอกสารอ้างอิง

Arcuri, A. (2018). Test suite generation with the Many Independent Objective (MIO) algorithm.

Information and Software Technology, 104, 195–206.

<https://doi.org/10.1016/j.infsof.2018.05.003>

Defects4J contributors. (n.d.). *Defects4J (Version 3.0.1-7-g8c16da82)* [Computer software]. GitHub.

<https://github.com/rjust/defects4j/tree/8c16da8230843cdc918eaf4ddb449637f02b83c6>

Fraser, G., & Arcuri, A. (2011). EvoSuite: Automatic test suite generation for object-oriented software. In

Proceedings of the 19th ACM SIGSOFT Symposium on the Foundations of Software

Engineering (pp. 416–419). <https://doi.org/10.1145/2025113.2025179>

Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled

testing studies for Java programs. In *Proceedings of the 2014 International Symposium on*

Software Testing and Analysis (pp. 437–440). <https://doi.org/10.1145/2610384.2628055>

Lei, Y., & Tai, K. C. (1998). In-parameter-order: A test generation strategy for pairwise testing. In

Proceedings of the 3rd IEEE High-Assurance Systems Engineering Symposium (pp. 254–261).

<https://doi.org/10.1109/HASE.1998.731623>

ภาคผนวก ก แหล่งข้อมูลและไฟล์ทำซ้ำ

เนื้อหา	ไฟล์หลัก
โจทย์รายวิชา	SQA_Project_2026 (1).pdf หน้า 1–3
Catalog	target_benchmark/all_bugs_catalog.json; target_benchmark/catalog_17_projects.json
Benchmark results	results/master_benchmark_summary.csv; results/run_logs/
Suite gap audit	results/suite_gap_audit.csv; scripts/audit_suite_gaps.py
สถิติและ analytics	results/master_descriptive_stats.json; results/advanced_analytics_report.md
Native IPO	Combinatorial_IPO/Code/; Combinatorial_IPO/Results/verified_suites_manifest.json; generation_manifest.json; routing_manifest.json
MIO	MIO_Algorithm/Code/batch_evosuite.py; MIO_Algorithm/Result_Round2/evosuite_budget_summary.csv; MIO_Algorithm/MIO_FAILURE_ANALYSIS_REPORT.md
AI prompts/economics	Deepseek-v4_flash/Prompt/; Gemini-3_8_flash/Prompt/; results/Deepseek_vs_Gemini_Economics.csv
Runner/analytics	scripts/run_benchmark.py; scripts/run_member3_delivery.py; scripts/consolidate_master_results.py; scripts/advanced_data_analytics.py; results/member3_delivery_logs/

ตารางที่ 11 ไฟล์หลักสำหรับตรวจสอบและทำซ้ำ

ภาคผนวก ข ตัวอย่างการตรวจย้อนกลับ

เทคนิค	Bug	Run ID	Test file	SHA-256 prefix
Native IPO	Chart-14	Chart-14-ipo-1790351771	XYPlot_IPOTest.java	a8319aa106c1278a...
MIO ใน EvoSuite	Jsoup-14	Jsoup-14-mio-1790379300	TokeniserState_ESTest.java;Tokeniser_ESTest.java	5da9081d476ce5f3...
DeepSeek V4 Flash	Closure-105	Closure-105-deepseek-1790360926	FoldConstantsDeepseekTest.java	4c51b036313f281c...
Gemini 3.8 Flash	Chart-3	Chart-3-gemini-1790350744	TimeSeriesGeminiTest.java	bcce42fcb234af77...

ตารางที่ 12 ตัวอย่าง BUG_DETECTED ที่ตรวจ buggy/fixed และ provenance แล้ว

ทั้งสี่ตัวอย่างมี buggy failures และไม่มี fixed failures ใน structured run log. Hash เต็มและรายการ failure อยู่ใน Run_Log ที่อ้างด้วย project + bug + technique ใน master CSV.

ภาคผนวก ค Data dictionary แบบย่อ

ฟิลด์	ความหมาย
Project, Bug_ID, Technique	คีย์หนึ่งการประเมินต่อบั๊กและเทคนิค
Target_Classes	modified target classes ในขอบเขตบั๊ก
Suite_Available, Test_Files	การมี suite และชื่อไฟล์ที่ประเมิน
Line_Coverage_%, Branch_Coverage_%	ค่าจาก Defects4J summary; วางเมื่อไม่ได้วัด
Fault_Detection_Status	BUG_DETECTED, NOT_DETECTED, FLAKY_OR_REGRESSION, COMPILE_ERROR หรือ NOT_EVALUATED
Execution_Status	สถานะ benchmark; สถานะ generation อยู่ใน audit; hash และ log อยู่ใน master

ตารางที่ 13 ความหมายฟิลด์สำหรับตรวจย้อนกลับ